

Token Accounting in Agentic Applications

General formulas for current context, deliberation output, multi-call turns, caching, estimation, and cost

Purpose: This document defines implementation-neutral token calculations. It does not assume any specific model vendor, agent framework, or user interface.

1. Scope and notation

An agentic application may call a language model several times during one user turn. Each call has its own input context and generated output. A user interface may show either the latest call's context, a cumulative turn total, or an estimate. These quantities must be named separately.

Symbol	Meaning
j	Index of a model invocation.
m	Number of model invocations in one agent turn.
$T_{in,j}$	Input tokens processed by invocation j .
$T_{out,j}$	Output tokens generated by invocation j .
$B(t)$	Bottom/status-bar token value at time t .
$D(t)$	Live deliberation-phase token value at time t .
$T\text{-hat}$	Estimated token count.

2. Input tokens for one model invocation

The input context is the complete tokenized payload supplied to one model invocation. It is the sum of all included context categories.

$$T_{in,j} = \sum_{c \in C} T_{c,j}$$

General category-sum form.

A practical expanded form is:

$$T_{in,j} = T_{system,j} + T_{instructions,j} + T_{history,j} + T_{user,j} + T_{tool\ spec,j} + T_{retrieval,j} + T_{tool\ result,j} + T_{other,j}$$

The exact category names depend on the application.

Categories must be disjoint when they are added. For example, file content retrieved by a tool must not be counted again if it is already included inside the tool-result category.

3. Cache-aware input accounting

Some APIs partition input tokens by cache treatment. When the reported categories are mutually exclusive:

$$T_{in,j} = T_{uncached,j} + T_{cache\ write,j} + T_{cache\ read,j}$$

Cache use changes processing and often price, but cached tokens still occupy logical context. Do not subtract cache-read tokens when calculating context-window occupancy.

4. Bottom-right or status-bar token value

The most useful general definition is the input-context size of the latest active or latest completed model invocation:

$$B(t) = T_{in, j^*(t)}$$

Here, $j^(t)$ is the invocation represented by the interface at time t .*

This is a current-context metric, not a session-total metric. An implementation may instead show an estimate, a cumulative turn total, or a window percentage; therefore the UI label should state which definition it uses.

Recommended label: “Current context tokens” for $B(t)$. Avoid the ambiguous label “Total tokens” unless the value is truly cumulative.

5. Output tokens for one invocation

Generated output may contain internal reasoning, visible assistant text, structured tool-call arguments, and other model-generated fields.

$$T_{out,j} = T_{reasoning,j} + T_{visible,j} + T_{tool\ call,j} + T_{other\ output,j}$$

Only include categories that the model API counts as output. If hidden reasoning is not separately exposed, use the authoritative aggregate output-token field.

6. Deliberating-phase token calculation

A live deliberation counter should accumulate output generated during the set of model calls belonging to the current phase.

$$D(t) = \sum_{j \in \mathcal{A}(t)} T_{\text{out},j}(t)$$

$\mathcal{A}(t)$ is the set of active or already-started calls included in the phase.

After the phase ends, the authoritative final value is:

$$D_{\text{final}} = \sum_{j \in \mathcal{P}} T_{\text{out},j}$$

$\mathcal{P}$ is the set of model invocations assigned to the deliberation phase.

For a single-call phase, this reduces to the output-token count of that call. For a multi-call agent loop, sum the output tokens of every call included in the phase.

7. Cumulative token use for one agent turn

A user turn may trigger planning, tool use, observation, revision, and final-answer calls. Cumulative turn usage sums over all model invocations.

$$T_{\text{turn},\text{in}} = \sum_{j=1}^m T_{\text{in},j}$$

$$T_{\text{turn},\text{out}} = \sum_{j=1}^m T_{\text{out},j}$$

$$T_{\text{turn},\text{total}} = T_{\text{turn},\text{in}} + T_{\text{turn},\text{out}}$$

This cumulative quantity is useful for telemetry and cost analysis. It is not the same as current context-window occupancy.

8. Context growth between calls

The next invocation usually retains some previous context and adds new material. Truncation, compaction, or context editing may remove material.

$$T_{\text{in},j+1} = T_{\text{retained},j} + T_{\text{new user},j+1} + T_{\text{new tools},j+1} + T_{\text{new retrieval},j+1} - T_{\text{removed},j+1}$$

$$T_{\text{removed}} = T_{\text{truncated}} + T_{\text{compacted}} + T_{\text{discarded}}$$

Because retained output and tool results can be rewritten, summarized, or dropped, the next context is not generally equal to the previous context plus the previous output.

9. Context-window percentage and free space

$$P_{\text{used}} = 100 \cdot \frac{T_{\text{context}}}{T_{\text{window}}}$$

$$T_{\text{free}} = T_{\text{window}} - T_{\text{context}} - T_{\text{reserved}}$$

Reserved space may be kept for generation, safety margins, or automatic compaction. The application must document whether the reserved amount is included in or excluded from the displayed used percentage.

10. Live estimation when exact usage is unavailable

During streaming, an exact tokenizer count may not yet be available. A client may estimate from character count:

$$\hat{T} \approx \frac{N_{\text{characters}}}{\kappa}$$

Here, κ is an empirically chosen average number of characters per token for the language and content type. This estimate can be poor for code, non-Latin scripts, whitespace-heavy text, or structured data.

$$E_{\text{rel}} = 100 \cdot \frac{|T - \hat{T}|}{T}$$

Replace the estimate with the API's final usage values as soon as they become available.

11. Display rounding

$$T_{\text{display}} = s \cdot \text{round}\left(\frac{T}{s}\right)$$

Here, s is the display step. For example, $s = 100$ gives rounding to the nearest hundred tokens. The raw integer should remain available in telemetry even when the UI shows an abbreviated value.

12. Cost calculation

Token counts and monetary cost are related but not identical. A general cache-aware cost formula is:

$$C = \sum_j (p_{\text{in}} T_{\text{uncached},j} + p_{\text{cw}} T_{\text{cache write},j} + p_{\text{cr}} T_{\text{cache read},j} + p_{\text{out}} T_{\text{out},j})$$

The p terms are prices per token for uncached input, cache writes, cache reads, and output. Use the provider's billing units and rates. If a category does not exist, set its term to zero.

13. Recommended implementation definitions

Metric	Definition
Current context	Use $B(t) = T_{\mathrm{in},j^*(t)}$.
Live deliberation	Use $D(t) = \sum T_{\mathrm{out},j}(t)$ over calls in the active phase.
Final deliberation	Replace estimates with $(\sum T_{\mathrm{out},j})$ from final API usage records.
Turn usage	Use separate cumulative input and output sums across every invocation.
Window occupancy	Use logical context tokens, including cache-read tokens.
Telemetry	Store raw integers, invocation IDs, phase IDs, timestamps, and whether each value is exact or estimated.

14. Common errors to avoid

- Adding a subset twice, such as adding file-read tokens to messages when file reads are already inside messages.
- Treating current context as cumulative session usage.
- Adding live output directly to the current input and calling the result the next context.
- Excluding cache-read tokens from context-window occupancy.
- Using a character-based estimate as an authoritative final count.
- Summing repeated streaming usage snapshots instead of taking the final value for each invocation.
- Mixing billing tokens, tokenizer tokens, and UI estimates without naming the source of each value.

15. Minimal reference formulas

Current input context:

$$B(t) = T_{\mathrm{in}, j^*(t)}$$

Live deliberation output:

$$D(t) = \sum_{j \in \mathcal{A}(t)} T_{\mathrm{out},j}(t)$$

Final multi-call deliberation output:

$$D_{\text{final}} = \sum_{j \in \mathcal{P}} T_{\text{out},j}$$

Cumulative agent-turn usage:

$$T_{\text{turn, total}} = T_{\text{turn, in}} + T_{\text{turn, out}}$$

These four formulas cover the most common status-bar, deliberation, and telemetry calculations in an agentic application.